

Dynamic Local Usage: An accurate model for usage of tile-internal wiring in Global Routing

Tilman Bihler

Research Institute for Discrete Mathematics
University of Bonn
Bonn, Germany
bihler@dm.uni-bonn.de

Daniel Blankenburg

Research Institute for Discrete Mathematics
University of Bonn
Bonn, Germany
blankenburg@dm.uni-bonn.de

Abstract—Traditional global routing simplifies routing as a Steiner Tree packing problem on a grid graph, often neglecting the need to account for tile-internal wiring connections to pin shapes. Local wiring usage is typically pre-estimated, leading to inaccuracies. To address this gap, we introduce the Dynamic Local Usage (DLU) model, which evaluates wiring usage based on exact wire shapes and optimizes routes up to detailed pin shapes. Key innovations include tip-to-tip penalty extensions to accurately model packing density and new algorithms to optimize routes according to the DLU congestion model. We demonstrate the superiority of the Dynamic Local Usage model by comparing results after global and detailed routing to an industrial global router on the 3nm and 5nm technology nodes. Our model achieves significant improvements in the number of vias, scenic routes, design rule violations, and timing metrics after detailed routing. Furthermore, the DLU provides a better support for incremental design changes, such as pin movements, by dynamically updating local wiring usage during routing.

Index Terms—Physical Design, VLSI, Placement and routing

I. INTRODUCTION

Routing in VLSI design is tasked with the computation of rectilinear Steiner trees that have to obey numerous constraints and design rules. In practice different objectives should be optimized and timing constraints need to be met. Due to its high complexity the routing process is traditionally split into at least two steps: *global routing* and *detailed routing*. In global routing the task is simplified by introducing a rectangular tiling that covers the chip area. Tile centers of adjacent tiles are connected, resulting in a grid graph. We assume that the chip area consists of L layers, each having a preferred dimension (x or y) in which wires must be routed. Such a grid graph is depicted in Fig. 1.

Traditionally, all pre-existing wires (prewires) and pins are projected onto the centers of these tiles, and Steiner trees are constructed on the global routing grid graph. This approach formulates the global routing problem as a Steiner tree packing problem, which often overlooks the necessity of connecting to the exact pin shapes. Consequently, the usage caused by tile-internal wiring is typically estimated prior to global routing and remains unadjusted based on the actual global routing solution [1]–[3]. While there is an extensive body of work dedicated to developing algorithms for solving the global routing problem (see surveys [4], [5]), relatively few studies focus on modeling capacity and wiring usage accurately. An improved model for local wiring usage offers several significant advantages:

- **Enhanced Input for Detailed Routing:** A more precise usage model provides the detailed router with better guidance, leading to higher-quality routing solutions.
- **Support for Incremental Design Changes.** During timing optimization based on global wires, small adjustments—such as slight pin movements or buffer insertions—are often made to an already globally routed design. These changes predominantly

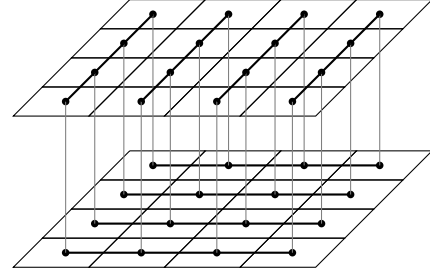


Fig. 1: A global routing grid graph with $L = 2$ layers

affect tile-internal wiring usage and may go unnoticed if local usages are estimated only once prior to routing.

- **Improved Scalability:** By accurately modeling tile-internal wiring usage, larger global routing tile sizes can be utilized without significantly degrading solution quality [6].

A. Our contribution

In this work, we introduce the Dynamic Local Usage model, a novel approach that dynamically adjusts local wiring usage based on the actual routing solution. Our key contributions are:

- 1) **An Intuitive Area-Based Capacity and Usage Model:** We present a capacity and usage model that applies to all wires and vias, enhanced by tip-to-tip penalty extensions. These extensions accurately model the packing inefficiencies inherent in detailed routing, leading to a more precise congestion assessment.
- 2) **A set of novel RC-aware algorithms on exact wire shapes:** These algorithms incorporate the tip-to-tip penalty extensions and improve delays on the global routes significantly.
- 3) **Superior routing results:** Through extensive experimentation (Table II, Table I), we demonstrate that our DLU model outperforms a state-of-the-art industrial global router in terms of key metrics after detailed routing, including wirelength, via count, detour factors (scenics), violation counts, and timing metrics.

Given the significant benefits of the Dynamic Local Usage model, DLU has been adopted as the default setting in our industrial VLSI routing toolkit, highlighting its practical impact and effectiveness in modern design environments.

II. THE DYNAMIC LOCAL USAGE MODEL

A. Area usages

a) **Edge capacities:** Let $e = \{u, v\}$ be an edge in the global routing graph. The edge area of e , denoted as $\text{area}(e)$, is the rectangle spanned by u and v , extending to the tile borders in the

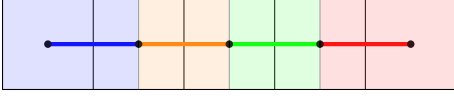


Fig. 2: Global routing edges e, f, g, h in a tile corridor and their corresponding edge areas

non-preferred direction, and further to the chip border if u or v are boundary nodes (Fig. 2). The capacity of e is determined as the total unblocked default track length available in $\text{area}(e)$ multiplied with the default wiring pitch p_z (width plus spacing of the default wiretype) on the layer z of edge e .

$$\text{cap}(e) = \text{unblocked_track_length}(\text{area}(e)) \cdot p_z \quad (1)$$

To compute the unblocked default track length, the exact positions and spacing requirements of all fixed objects (pins, blockages, and locked input wires) within $\text{area}(e)$ are considered, and the blocked track length is subtracted from the total default track length in $\text{area}(e)$. When tracks are tightly packed and no fixed objects are present, $\text{cap}(e)$ equals the total area of $\text{area}(e)$.

b) Area usage of a Steiner tree: We proceed to define the area usage of a Steiner tree Y from edge e . Let $w \in E(Y)$ be a wire or via. Let us denote the rectangular shape of the wire or viapad by $\text{shape}(w)$. It would be natural to compute the area usage based on the union of shapes of Y , however this is too optimistic. We define a *unit space usage*, denoted by $\text{unit_usage}(w)$ (see Section II-B) for every wiretype and viapad on every layer. Let $\text{usage_shape}(w)$ be the rectangle that has the same centerline as $\text{shape}(w)$ in preferred dimension and a width of $\text{unit_usage}(w)$ in non-preferred dimension. Again, simply computing the usage on the area of the usage shapes would be too optimistic, due to the heavy simplification of design rules in global routing. Instead, we must account for the inevitable increases in vias (10–20%) and wirelength (5%) during detailed routing (see Table II, Table I). With Dynamic Local Usage, we take a precise approach by introducing *tip-to-tip penalty extensions*, which account for packing inefficiencies and wire spreading during the transition to detailed routing. These tip-to-tip penalty extension (see Section II-C) are applied to wire ends and via shapes in preferred direction, see Fig. 5. Let $\text{t2t_usage_shape}(w)$ be $\text{usage_shape}(w)$ extended by the tip-to-tip penalty extension (defined in Section II-C) in positive and negative preferred direction.

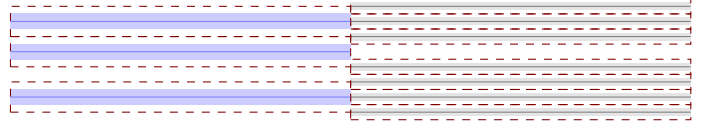
Finally, we define the usage of Y from e as the intersection area of the union of all the tip-to-tip extended usage shapes of Y with $\text{area}(e)$, divided by the capacity of e .

$$\text{usg}_Y(e) := \frac{|\text{area}(e) \cap \bigcup_{w \in E(Y)} \text{t2t_usage_shape}(w)|}{\text{cap}(e)} \quad (2)$$

B. Non-default wire usage

The capacity of an edge is measured as the available default track length times the default wiring pitch. Naturally, the unit space usage of a default wire w on layer z is given by the default pitch on layer z . However, non-default wiretypes, which are not necessarily placed on the default track pattern, require special consideration. A naive approach is to measure the usage by comparing the wire pitch of the non-default wiretype and the default wiretype. This neglects that in many cases, non-default track patterns can be defined in a more sophisticated way such that they have fewer conflicts with the default track pattern.

When specific track patterns for non-default wiretypes are available (as in our experiments), we compute their unit space usage by



(a) Track patterns for default (gray) and double-wide (blue) wiretypes are shown, with spacing constraints in red. In the worst case, a blue wire blocks three default tracks, while on average, it blocks $\frac{7}{3}$ tracks. This gives a unit space usage of $\text{unit_usage}(w) = 0.5 \times (3 + \frac{7}{3})p_z = \frac{8}{3}p_z$.



(b) Transformation of the shape of w into its usage shape.

Fig. 3: Illustration of the usage shape computation for non-default wiretypes.

taking the average between the worst-case and average-case scenarios regarding the number of default tracks they block. This is illustrated in Fig. 3a. Similarly, the unit space usages of via pad shapes are computed by considering the number of blocked default tracks. As described in Section II-A, we compute usage shapes based on the unit space usages, as illustrated in Fig. 3b.

A detailed overview on estimation of wire usages based on track patterns can be found in [7].

C. Tip-to-tip penalty extensions

Tip-to-tip penalty extensions model two key effects observed during detailed routing:

- *Packing inefficiencies:* Gaps larger than the minimum tip-to-tip spacing are left between wires due to the complexity of detailed routing.
- *Spreading of wires:* Overlapping wires after global routing must be spread to different tracks in detailed routing, leading to elongation of adjacent wires on neighboring layers and increased space usage near route bends.

To address these effects, we extend $\text{usage_shape}(w)$ of a wire w with wiretype t on layer z on both ends by some length $\text{penalty}_{z,t} > 0$ and calculate the area usage of the resulting extended shape $\text{t2t_usage_shape}(w)$ as discussed in Section II-A. We proceed analogously for via pad shapes and stacked vias. If a via pad shape has a larger unit space usage than the corresponding wire shape (a so-called *clubfoot via*), we extend the via pad shape in both positive and negative preferred direction by $\text{penalty}_{z,t}$, as shown in Fig. 5. Fig. 4 shows the planar projection of a global route with tip-to-tip penalty extensions. The penalty length is determined by:

$$\text{penalty}_{z,t} := \min \left\{ \begin{array}{l} \frac{k_{\text{tip}}}{2} \cdot \text{t2t}_{z,t}, \\ \frac{k_{\text{tip}}}{2} \cdot 0.6 \cdot (p_{z+1,t} + p_{z-1,t}), \\ 0.4 \cdot \text{length}_{\text{tile}} \end{array} \right\}, \quad (3)$$

where

- $\text{t2t}_{z,t}$ is the minimum required spacing in preferred dimension of two wires with wiretype t (*tip-to-tip spacing*) on layer z ,
- $p_{z,t}$ is the *pitch*, i.e. the minimum required distance between two tracks, for wiretype t on layer z ,
- $\text{length}_{\text{tile}}$ denotes the average length of a global routing tile,
- $k_{\text{tip}} \geq 1$ is a tunable penalty parameter dependent on tile size, technology node, and the detailed router's packing efficiency.

We discuss our choice of parameters in Section IV. (3) incorporates the following:

- *Minimum required tip-to-tip spacing:* Note that $\frac{\text{t2t}_{z,t}}{2}$ is the minimum required area usage that should be charged at the

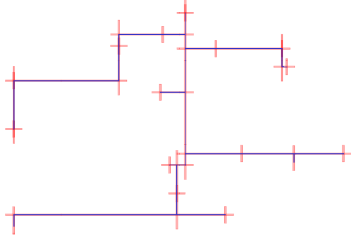


Fig. 4: A global route with tip-to-tip penalty extensions

end of a wire. Indeed, if two wires are abutting, they require a minimum spacing of $\tau_2 \tau_{z,t}$ between them, and it is natural to account for half of this space usage at each of the wires.

- *Pitch of adjacent layers:* These affect the spreading on adjacent layers and thus the potential extension of the wire segment on layer z during detailed routing.
- *Tiles size bound:* The last term prevents overcongestion on higher layers where tip-to-tip spacing becomes large due to thicker wires.



Fig. 5: Illustration of the union of a wire usage shape with the usage shape of a clubfoot via pad (blue) with tip-to-tip penalty extensions (red) applied at both ends of the usage shapes. The area usage of the wire and the clubfoot via is computed according to the union of the grey and the red rectangles.

III. ALGORITHMS

A. The global routing algorithm

We formulate the fractional relaxation of the global routing problem as a resource sharing problem and solve it approximately with the algorithm described in [8]. The resource sharing algorithm proceeds in several phases and computes multiple trees for each net. With DLU, also the local wiring is optimized and recomputed in every tree computation. After that we apply randomized rounding to obtain an integral solution and conclude with a post-optimization, that is similar to typical ripup & reroute approaches. Our entire algorithmic framework is timing-aware. The resource sharing formulation includes timing constraints as described in [9] and our Steiner tree approximation algorithm follows a similar approach to [10]. Let $\mathcal{R}$ denote the set of resources, including edges in the global routing graph, netlength resources, and timing resources for every sink in each net. To ensure convergence to an optimal solution, the Steiner tree approximation algorithm has to find a tree Y that minimizes the total cost:

$$\text{cost}(Y) = \sum_{r \in \mathcal{R}} \text{price}(r) \text{usg}_Y(r) \quad (4)$$

As in most global routing algorithms the price of resource r , $\text{price}(r)$, is exponential in the load of r . The usage $\text{usg}_Y(r)$ represents the area usage for edge resources (as discussed in Section II-A) and the RC-delay for sink resources within tree Y .

Within the dynamic local usage model the tree Y on which we measure the usages is connected to the exact pin shapes. Our algorithm to find such a tree proceeds in three steps:

- 1) RC-aware Steiner tree algorithm on the global routing graph (Section III-B),
- 2) RC-aware optimization of x - and y -coordinates within tiles (Section III-C),
- 3) RC-aware layer embedding algorithm (Section III-D).

B. Finding tile-center to tile-center routes

We employ a Steiner tree approximation algorithm with a Prim-like path decomposition heuristic on the three-dimensional global routing grid graph. To handle RC-delays, we adopt techniques similar to those in [9], [10], which involve splitting certain components and reconnecting them to the source pin via RC-aware path searches.

The inclusion of tip-to-tip penalty extensions introduces additional costs at vias in the route, making the cost dependent on the route's structure. This complexity means that considering only edge costs in the path search algorithm is insufficient. To address this, we model the problem as a Steiner tree problem with node costs. More precisely, for a node v with incident preferred-dimension edges e and f , the node cost is the sum of the tip-to-tip penalty extension costs on e and f .

We adjust the edge costs of a preferred-dimension edge e by subtracting twice the tip-to-tip penalty cost from its original cost, ensuring the result is non-negative by taking the maximum with zero. For via edges, we charge congestion costs only for the tip-to-tip penalties caused by overhanging via pad shapes. Other cost components, such as those for timing or netlength resources, remain unchanged. Given that the maximum degree of a node in the global routing graph is 4 and assuming that the tip to tip penalty costs are small compared to the edge costs, this can be shown to yield a close-to 3-approximation for this Steiner tree problem with node costs, as shown in [6].

C. RC-aware XY-optimization

Given the tile-center to tile-center route, we first realise tile-internal connections to the pin shapes with a rectilinear minimum Steiner tree routine.

We then optimize the x - and y -positions of the Steiner nodes within their tiles by considering the two-dimensional projection of the route. Our algorithm employs an improved version of the algorithm from [11] and incorporates:

- *Congestions costs:* In the presence of area usages, tile-internal movement of nodes affects the congestion costs. To account for this in the planar case, we consider the worst congestion over the assigned layers which follow the preferred dimension of a segment.
- *RC-Delays:* We include RC-delays heuristically by estimating the downstream capacitance of a node by assuming the minimum capacitance over all layers for all segments in the subtree.

The algorithm optimizes x - and y -coordinates independently and proceeds as a dynamic program in reverse topological order on Y , propagating piecewise linear functions for the subtree costs. For a tree Y our algorithm has running time $\mathcal{O}(|V(Y)| \cdot m)$, where m is the maximum number of breakpoints occurring in the propagation of the piecewise linear functions. In the worst case the number of breakpoints can be linear in $|V(Y)|$, but has been shown to be much smaller in practice experimentally [6].

D. RC-aware layer embedding

We assume that the input to the layer embedding algorithm is a source-rooted planar arborescence, possibly containing tile-internal diagonal arcs between nodes. Our layer embedding supports these

Input: A source-rooted planar arborescence Y .

Output: A layer embedding of $E(Y)$.

```

1 for each edge  $e = (u, v)$  in  $Y$  in reverse topological order do
2   Let  $\delta_Y^+(v)$  be the set of outgoing edges from  $v$ .
3   for each ordering  $e_1, e_2, \dots, e_k$  of  $\delta_Y^+(v)$  do
4     // Fixing the ordering allows us to approximate
4     // RC-delay costs of connecting via edges
5     for each possible layer  $l \in \{1, \dots, L\}$  do
6       Compute the minimal cost of assigning layer  $l$  to
6       // edge  $e$ , considering:
7       - The minimal costs of child edges  $e_i$ , assigned
7       // to layers compatible with  $l$ , following the fixed
7       // ordering.
8       - Congestion and RC-delay costs of edge  $e$  and
8       // connecting via edges, based on approximated
8       // downstream capacitance from child edges.
9       Store the minimal cost and corresponding layer
9       // assignments of the child edges.
10    end
11  end
12 end
13 return the minimal cost assignment at the root edge.

```

Algorithm 1: The layer embedding dynamic program

diagonal arcs by selecting the cheaper of the two possible L-shaped realizations for each. We split segments at edge area borders to ensure we find an optimal congestion-aware solution.

Dynamic programming is a standard technique for layer embedding, particularly in global routing methods that separate 2D routing from layer assignment [12], [13]. The layer embedding algorithm of [14] also considers RC-delays and congestion costs in a dynamic program, albeit without stating a runtime or bounds on the solution quality.

Similar to these approaches, our algorithm traverses the tree in reverse topological order, embedding segments while considering the optimal embeddings of child arcs in the subtree.

Key distinctions of our algorithm include:

- **Embedding exact wire shapes and considering tip-to-tip penalty extensions.**
- **Fast estimates for RC-costs:** We estimate downstream capacitance by assuming minimum wire capacitances for all wires in the subtree, and using a correction factor based on a lower bound on upstream resistance, similar to the method for RC-aware path search in [10].

Incorporating RC costs adds complexity because the order in which child branches are embedded affects the delay costs of vias.

Our algorithm achieves optimal embeddings in the timing-unaware setting and provides a good a posteriori approximation ratio in the RC-aware setting. It is described in pseudo-code as Algorithm 1.

Since nodes in the planar tree have a bounded degree, we can feasibly explore all possible orderings of child embeddings, that is, the for-loop in Line 3 of Algorithm 1 has a constant number of iterations. It is an improved version of the algorithm described in [15], using a very similar definition of RC-aware costs, improving on the running time by a factor of L . The key insight that leads to the running time improvement is that for a fixed ordering, the minimal cost assignments can be computed recursively bottom-up and top-down for all layers, leading to an accumulated runtime of $\mathcal{O}(L)$ for the for-loop in Line 5 of Algorithm 1. Details can be found in [6].

Theorem 1: Algorithm 1 computes a $(1 + (\alpha - 1)(1 - \beta))$ -approximately optimal solution within a running time of $\mathcal{O}(|E(Y)| \cdot L)$. Here $\alpha \leq c_{\max}/c_{\min}$ is an upper bound on our downstream capacitance estimate error and $\beta \geq r_{\min}/r_{\max}$ is a lower bound on our upstream resistance estimate error. The values $c_{\min}$, $c_{\max}$, $r_{\min}$, $r_{\max}$ denote the minimum and maximum wire capacitance, respective resistance across all layers.

Note that α is an upper bound on the approximation ratio and $\alpha \leq c_{\max}/c_{\min}$ holds. This ratio is close to 1 in recent technologies. Details can be found in [6].

IV. RESULTS

A. Test setup

All experiments were conducted on a 2×64 AMD EPYC 7742 CPU machine with 2TB of RAM, utilizing 32 threads. The Dynamic Local Usage model was implemented within an industrial VLSI design toolkit. For comparison, results were benchmarked against a traditional global router from the same toolkit by running the same industrial detailed router on the global solutions produced by both models.

The traditional global router is also RC-aware and follows a similar methodology to the one described in Section III, albeit employing simpler pin connection algorithms.

The timing-unaware experiments were performed on a combined testbed of 95 state-of-the-art microprocessor units from the 3nm and 5nm technology nodes. Meanwhile, the timing-aware experiments were done on a testbed of 17 state-of-the-art microprocessor units from the 5nm technology node. All test designs were extracted from an industrial design environment prior to routing.

For the experiments using the DLU model, a tip-to-tip penalty factor $k_{tip} = 14.5$ was set for the 5nm designs and $k_{tip} = 13$ for the 3nm designs. All runs were conducted with a global routing tilesize of about 70 times the minimum wiring pitch across all layers.

B. Experimental results in a timing-unaware routing flow

a) Metrics: To evaluate the efficacy of the DLU model in guiding detailed routing, we compared its performance to that of the traditional global router (trad) across several metrics on the mixed testbed of 95 designs from the 3nm and 5nm technology nodes. The measured metrics in Table II include:

- **Wirelength and Vias:** Total wirelength and the number of vias after global routing (GR WL, GR Vias) and detailed routing (DR WL, DR Vias).
- **Global Routing Runtime:** The total runtime of global routing (GR Walltime).
- **Violations:** The total number of design rule violations (DRVs) as well as the number of wires and vias causing one or more spacing violations (Violation Wires, Violation Vias).
- **Scenics:** The number of nets that were routed 25%, 50%, and 100% longer than a shortest rectilinear Steiner tree after detailed routing (DR Scenics 25, 50, 100).

b) Results and Discussion: Table II demonstrates that the DLU model shows significant improvements over the traditional global router:

- **Wirelength and Vias:** A substantial reduction in the number of vias and a wirelength were observed after global routing. These improvements stem from the more accurate congestion model and the optimized local wiring strategies discussed in Section III. The traditional router also computes connections to the local pin

Chip nets	Run	GR WL (m)	GR Vias	wACE4	GR TnS (ps)	GR WS (ps)	GR Walltime	DR WL (m)	DR Vias	DRVs	Violation Wires	Violation Vias	DR WS (ps)	DR TnS (ps)
U_1	trad	0.0402	57651	85.99 %	-7762	-73	00:00:21	0.0432	58557	0	0	0	-75	-8529
7096	DLU	0.0403	54437	86.75 %	-7278	-71	00:00:32	0.0432	58238	3	1	1	-76	-8637
U_2	trad	6.4131	3339916	90.08 %	-105472	-147	00:09:17	6.6953	3469906	175	3	2	-164	-233517
325295	DLU	6.3173	3178308	90.27 %	-75443	-112	00:17:42	6.6010	3479066	187	0	0	-118	-203004
U_3	trad	12.2375	8126263	88.21 %	-365071	-84	00:22:16	12.9010	8538964	248	30	28	-100	-708008
815646	DLU	12.1876	7561152	90.24 %	-258110	-84	00:32:41	12.8604	8574487	170	2	5	-85	-703788
U_4	trad	2.1136	1475502	88.37 %	-28233	-48	00:05:27	2.1930	1537545	1155	0	0	-67	-72586
188249	DLU	2.1086	1432395	82.75 %	-22240	-43	00:07:32	2.1905	1537156	1147	0	0	-68	-70606
U_5	trad	0.7596	545084	79.80 %	-18	-4	00:02:06	0.7887	558179	179	0	0	-18	-173
63620	DLU	0.7578	518403	89.28 %	-6	-5	00:05:26	0.7874	555995	179	0	0	-16	-134
U_6	trad	0.0760	83973	74.74 %	-19386	-162	00:00:24	0.0797	84595	74	0	0	-181	-19890
11264	DLU	0.0758	77183	73.92 %	-19307	-162	00:00:31	0.0796	83232	79	0	0	-177	-19802
U_7	trad	1.9506	1797534	88.81 %	-31950	-35	00:04:39	2.0744	1874937	53	0	27	-56	-83968
193659	DLU	1.9350	1683718	88.73 %	-24626	-31	00:06:23	2.0629	1881236	31	1	19	-55	-77767
U_8	trad	1.1089	1766946	87.50 %	-18724	-33	00:05:12	1.2048	2025030	16	0	0	-51	-51659
247022	DLU	1.1080	1787859	86.12 %	-12191	-33	00:08:49	1.2037	2005529	18	0	0	-54	-51356
U_9	trad	1.0524	2017272	87.31 %	-6916	-25	00:05:05	1.1434	1898388	21	0	0	-49	-34706
228521	DLU	1.0524	1665140	86.01 %	-4873	-24	00:08:29	1.1440	1888193	26	0	0	-49	-33219
U_{10}	trad	0.8398	676946	82.55 %	-5901	-35	00:02:21	0.8779	693251	98	1	4	-49	-12467
84125	DLU	0.8374	652625	83.90 %	-5050	-35	00:04:05	0.8765	693337	82	0	0	-50	-11785
U_{11}	trad	0.8707	862294	84.58 %	-4638	-24	00:01:59	0.9264	888493	19	0	0	-25	-5836
89263	DLU	0.8698	799497	85.61 %	-4530	-24	00:03:11	0.9251	882951	18	0	0	-24	-6153
U_{12}	trad	0.0570	61944	81.01 %	-14698	-549	00:00:20	0.0603	62356	29	0	0	-557	-15606
9250	DLU	0.0570	59695	81.13 %	-14346	-538	00:00:25	0.0602	62277	30	0	0	-557	-15564
U_{13}	trad	0.0007	882	55.81 %	0	46	00:00:11	0.0007	890	0	0	0	44	0
149	DLU	0.0007	842	48.65 %	0	44	00:00:11	0.0007	850	0	0	0	42	0
U_{14}	trad	0.4926	526766	86.54 %	-214177	-189	00:01:30	0.5212	562857	7	0	0	-197	-224122
68817	DLU	0.4902	487210	85.25 %	-210666	-189	00:02:32	0.5198	560112	16	0	0	-197	-224807
U_{15}	trad	0.0841	108281	83.33 %	-20841	-324	00:00:35	0.0893	112001	14	0	0	-326	-23033
14286	DLU	0.0834	103462	82.69 %	-20614	-322	00:00:54	0.0886	111373	14	0	0	-325	-23269
U_{16}	trad	0.4672	550596	86.75 %	-28736	-90	00:01:21	0.4963	562434	7	0	0	-98	-61724
67837	DLU	0.4655	511831	86.23 %	-20504	-86	00:02:01	0.4950	563610	9	0	0	-96	-63673
U_{17}	trad	1.1403	1196247	83.76 %	-21472	-93	00:02:53	1.1992	1220630	21	0	0	-47	-41588
144203	DLU	1.1359	1114358	83.83 %	-10116	-63	00:05:13	1.1966	1218459	30	0	0	-48	-45375
Summary	trad	29.7043	23594097		-893993		01:05:57	31.2949	24149013	2118	34	61		-1597411
	DLU	29.5227	21688115		-709900		01:46:37	31.1353	24156101	2039	4	25		-1558941
$\Delta\%$		-0.61 %	-8.08 %		-20.59 %		+61.66 %	-0.51 %	+0.03 %	-4 %	-88.24 %	-59.02 %		-2.41 %

TABLE I: Results in a timing-aware routing flow after global and detailed routing

	Metric	Traditional	Dynamic Local Usage
	GR Wire length	505.7703m m	477.7347 m -4.84 %
	DR Wire length	498.2275 m	497.2327 m -0.20 %
	GR Vias	288584789	252005360 -12.19 %
	DR Vias	301822488	300595525 -0.41 %
	GR Walltime (hh:mm:ss)	08:23:29	10:13:48 +24.12 %
	DRVs	13555	12448 -8.17 %
	Violation Wires	1418	1293 -8.82 %
	Violation Vias	3593	2920 -18.73 %
	DR Scenics 25	107229	60651 -43.44 %
	DR Scenics 50	17610	7126 -59.53 %
	DR Scenics 100	497	473 -4.83 %

TABLE II: Combined results from a timing-unaware routing flow on a testbed of 95 chips (default)

shapes, though in a very naive way. The reductions persisted into detailed routing, though with a smaller magnitude.

- *Violations*: We observe significant improvements in terms of violations, especially the number of violation vias decreases. This highlights the DLU model's capacity to provide a more better input to detailed routing.
- *Scenics*: The sharp decline in the number of DR Scenics indicates fewer routing detours, as the detailed router benefited

from an improved global solution.

However, the DLU model incurred a minor overhead in global routing runtime, due to the optimization of several tile-internal nets and the additional xy-optimization and layer embedding algorithms. Note that also tile-internal nets are recomputed several times across different phases in the resource sharing algorithm with DLU. This overhead is relatively minor when compared to the detailed routing runtime, which exceeded 50 hours for this testbed.

C. Experimental results in a timing-aware routing flow

We provide results after global routing and after detailed routing in a timing-aware flow in Table I.

a) *Metrics*: For the timing-aware experiments, we measured a subset of the metrics from Section IV-B, along with additional timing and congestion metrics:

- *Congestion (wACE4)*: A weighted average of the 5% most congested global routing edges, commonly used to assess routability [16]. While the wACE4 values from the two models are not directly comparable due to differences in usage and congestion calculations, they were similar in magnitude.
- *Timing Metrics*: Worst slack after global and detailed routing (GR WS, DR WS) and the total negative slack (GR TnS, DR TnS). Slacks were computed using an industrial signoff tool using RICE [17].

b) *Results and Discussion:* The timing-aware experiments yielded results consistent with the timing-unaware testbed, with additional timing-specific insights:

- *Wirelength, Vias and Violations* As in the timing-unaware flow, DLU achieved significant reductions in vias and wirelength. We point out that the global wirelength reduction is not as drastic as in Section IV-B, because the traditional global router optimizes the x- and y-positions of the pin connected routes to improve timing. Improvements in violations were particularly pronounced on unit U_3 , where the traditional router underestimated congestion, leading to poor detailed routing performance.
- *Timing Metrics:* Improvements in the FOM after global routing were substantial, driven by RC-aware algorithms described in Section III. Although the detailed router is timing-unaware, heuristic adherence to the global routing solution led to modest improvements in timing metrics post-detailed routing.

Despite the DLU model incurring a larger runtime increase in the timing-aware flow—attributed to the higher complexity of RC-aware optimizations—the overhead is justified by significant reductions in vias and improvements in slack metrics.

V. CONCLUSION

The Dynamic Local Usage (DLU) model enhances traditional grid-based global routing approaches by dynamically optimizing local wiring usage with exact wire shapes. Incorporating innovative features such as tip-to-tip penalty extensions and novel RC-aware algorithms to optimize local wiring with regard to the DLU model—including a Steiner tree approximation on the global routing graph, planar tile-internal optimization, and a layer embedding algorithm—DLU provides a more accurate input to detailed routing. Our experimental results demonstrate that DLU significantly improves the quality of detailed routing solutions across a variety of metrics, including via count, scenic routes, design rule violations, and timing metrics, while maintaining reasonable computational overhead. The adoption of DLU in our industrial toolkit highlights its practical impact.

REFERENCES

- [1] M. Gester, D. Müller, T. Nieberg, C. Panten, C. Schulte, and J. Vygen, “Bonnroute: Algorithms and data structures for fast and good vlsi routing,” *ACM Transactions on Design Automation of Electronic Systems (TODAES)*, vol. 18, no. 2, pp. 1–24, 2013.
- [2] Y. Wei, C. Sze, N. Viswanathan, Z. Li, C. J. Alpert, L. Reddy, A. D. Huber, G. E. Tellez, D. Keller, and S. S. Sapatnekar, “Glare: Global and local wiring aware routability evaluation,” in *Proceedings of the 49th Annual Design Automation Conference*, 2012, pp. 768–773.
- [3] J. He, U. Agarwal, Y. Yang, R. Manohar, and K. Pingali, “Sproute 2.0: A detailed-routability-driven deterministic parallel global router with soft capacity,” in *2022 27th Asia and South Pacific Design Automation Conference (ASP-DAC)*. IEEE, 2022, pp. 586–591.
- [4] J. Hu and S. S. Sapatnekar, “A survey on multi-net global routing for integrated circuits,” *Integration*, vol. 31, no. 1, pp. 1–49, 2001.
- [5] H. Tang, G. Liu, X. Chen, and N. Xiong, “A survey on steiner tree construction and global routing for vlsi design,” *IEEE Access*, vol. 8, pp. 68 593–68 622, 2020.
- [6] T. Bihler, “Dynamic local usage in bonnroute-global,” Ph.D. dissertation, University of Bonn, 2023.
- [7] P. Saccardi, “Continuous routing,” Ph.D. dissertation, University of Bonn, 2022.
- [8] D. Müller, K. Radke, and J. Vygen, “Faster min–max resource sharing in theory and practice,” *Mathematical Programming Computation*, vol. 3, no. 1, pp. 1–35, 2011.
- [9] S. Held, D. Müller, D. Rotter, R. Scheifele, V. Traub, and J. Vygen, “Global routing with timing constraints,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 37, no. 2, pp. 406–419, 2017.
- [10] R. Scheifele, “Rc-aware global routing,” in *2016 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*. IEEE, 2016, pp. 1–8.
- [11] A. K. Kiefner, “Minimizing path lengths in rectilinear steiner minimum trees with fixed topology,” *Operations Research Letters*, vol. 44, no. 6, pp. 835–838, 2016. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0167637716301481>
- [12] M. Pan, Y. Xu, Y. Zhang, and C. Chu, “Fastroute: An efficient and high-quality global router,” *VLSI Design*, vol. 2012, no. 1, p. 608362, 2012.
- [13] D. Shi, E. Tashjian, and A. Davoodi, “Dynamic planning of local congestion from varying-size vias for global routing layer assignment,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 36, no. 8, pp. 1301–1312, 2017.
- [14] G. Liu, X. Zhang, W. Guo, X. Huang, W.-H. Liu, K.-Y. Chao, and T.-C. Wang, “Timing-aware layer assignment for advanced process technologies considering via pillars,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 41, no. 6, pp. 1957–1970, 2022.
- [15] R. Scheifele, “Timing-constrained global routing with rc-aware steiner trees and routing based optimization,” Ph.D. dissertation, University of Bonn, 2019.
- [16] Y. Wei, C. Sze, N. Viswanathan, Z. Li, C. J. Alpert, L. Reddy, A. D. Huber, G. E. Tellez, D. Keller, and S. S. Sapatnekar, “Techniques for scalable and effective routability evaluation,” *ACM Trans. Des. Autom. Electron. Syst.*, vol. 19, no. 2, Mar. 2014. [Online]. Available: <https://doi.org/10.1145/2566663>
- [17] C. L. Ratzlaff and L. T. Pillage, “Rice: Rapid interconnect circuit evaluation using awe,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 13, no. 6, pp. 763–776, 1994.